Semantic Web
XX(X):1–17

DOI: 10.1177/ToBeAssigned
www.sagepub.com/
SAGE

DAIMO: An Ontology Profile for Governed AI Model Assets in Dataspaces

Edmundo de Elvira Mori Orrillo[1] and Jiayun Liu[1]

Abstract
Purpose: AI models are increasingly published, negotiated, deployed, invoked, monitored, and evaluated across organisational boundaries. Existing vocabularies describe important fragments of this lifecycle, but they do not connect catalog publication, policy, deployment, execution provenance, audit evidence, and evaluation context into one auditable governance chain. Methods: We present DAIMO, the Dataspace AI Model Ontology, an OWL 2 DL integration profile developed with the Linked Open Terms methodology. DAIMO organises established Semantic Web vocabularies around the missing governance relations between catalogues, policies, deployments, executions, provenance records, and evaluations. Results: The artefact is evaluated through an illustrative multi-participant dataspace scenario, OWL consistency and entailment verification, SHACL structural and cross-class validation, a negative test bench, and twenty-three competency questions executed as SPARQL queries. Conclusions: DAIMO provides a reusable semantic bridge for governed AI-model assets in dataspaces, making publication, discovery, invocation, traceability, and evaluation queryable and validatable as one coherent lifecycle.

Keywords
AI model governance, dataspace, ontology alignment, SHACL validation, LOT methodology

Introduction

Regulation (EU) 2024/1689, the AI Act, requires technical documentation, execution records, lifecycle traceability, and audit evidence for high-risk AI systems[1]. At the same time, European dataspaces are establishing an infrastructure in which autonomous organisations publish resources, negotiate contracts, and exchange data and services through connectors such as Eclipse Dataspace Components (EDC)[2] and protocol specifications such as the Dataspace Protocol (DSP)[3]. The intersection of these two developments creates a new object of semantic governance: an AI model as a federated asset whose description must support discovery, policy negotiation, authorised invocation, provenance reconstruction, and evaluation across organisational boundaries.

Semantic Web vocabularies already cover important fragments of this lifecycle. MLDCAT-AP 3.0.0[4] extends DCAT for machine-learning assets, representing models together with tasks, runs, evaluations, risks, and compute resources. DCAT[5] provides the cataloguing layer; ODRL[6] represents offers, permissions, prohibitions, and agreements; PROV-O[7] records provenance over entities, activities, and agents; and DSP specifies the contract-negotiation and transfer messages implemented by dataspace connectors. Narrative documentation artefacts such as model cards[8], factsheets[9], and datasheets[10] further improve human-facing transparency.

The remaining challenge is not the absence of vocabularies, but the absence of an explicit semantic bridge between them. A machine-learning model described with MLDCAT-AP is not, by that fact alone, an offering in a federated catalog. An ODRL offer can state permissions without being tied to the model and deployment it governs. A DSP negotiation records an exchange process, but not the AI-specific evaluation context or audit evidence that may condition lawful use. A PROV bundle can group provenance descriptions without distinguishing the cross-participant governance chain of an authorised model invocation. The modelling problem is therefore relational rather than terminological: the missing semantics live at the boundaries between cataloguing, policy, deployment, execution, provenance, and evaluation. Without those boundary terms, each lifecycle fragment remains describable, but the end-to-end governance chain remains difficult to query, validate, and audit.

This article presents DAIMO (Dataspace AI Model Ontology), an OWL 2 DL integration profile for governed AI-model assets in dataspaces. DAIMO follows a reuse-first design: it imports the semantics of established vocabularies where they already provide the required concept, and introduces a dedicated bridge construct only when the articulation between catalog, policy, execution, provenance, and evaluation cannot

[1]Department of Artificial Intelligence, ETSI Informáticos, Universidad Politécnica de Madrid, Spain

Corresponding author:
Edmundo de Elvira Mori Orrillo, Department of Artificial Intelligence, ETSI Informáticos, Universidad Politécnica de Madrid, Spain.
Email: edmundo.mori.orrillo@upm.es

otherwise be represented. The resulting contribution is threefold:

- a reuse-first ontology profile that preserves established vocabulary commitments while adding only the bridge constructs needed for governed AI-model exchange;
- a requirements and validation package that links five actor needs to twenty-three competency questions, formal consistency checks, entailment analysis, structural validation, and cross-class governance invariants;
- a reproducible multi-participant demonstrator that exercises publication, discovery, execution, auditability, evaluation, and governance-bridge queries while keeping the evidential claim bounded to executable modelling, not deployment validation.

The demonstrator uses five anonymised actor types: provider, consumer, platform operator, evaluator, and compliance actor. These actors instantiate the profile over a geospatial risk-prediction model. The demonstrator is not presented as deployment evidence; its purpose is to make the modelling commitments executable and inspectable. The article next reviews related work and positions the gap; it then describes the requirements and design decisions, presents the ontology, reports the evaluation, and discusses limitations, integration paths, and future work.

Background and related work

DAIMO is positioned at the intersection of three bodies of work: semantic descriptions of AI models, governance vocabularies and dataspace standards, and reusable ontology-engineering practices. We review these bodies in turn, focusing on the concepts that can be reused, the gaps that remain at their boundaries, and the evaluation expectations for an ontology intended to function as an operational profile. Table 2 then compares the relevant families of work against the five operational attributes that a profile for governed AI models must cover.

Describing AI models

Descriptions of AI artefacts appear at two levels of formality. Narrative documentation—model cards[8], factsheets[9], and datasheets for datasets[10]—raised transparency standards by describing intended use, limitations, benchmark behaviour, and training conditions in natural language. These formats are valuable for human readers and qualitative accountability, but insufficient as a substrate for automated discovery, policy-based filtering, or evidence linking in machine-to-machine flows.

Structured ontologies of the ML lifecycle cover the machine-consumable layer. EXPO[11] offers a general ontology of scientific experiments that later work has built on. ML-Schema[12] formalises machine-learning experiments as a reusable, lightweight schema. MEX[13] provides an interchange vocabulary, and OntoDM[14] together with DMOP[15] organise the data-mining domain. Souza et al.[16] and Schmidt et al.[17] extend the perspective towards provenance and FAIRness across the lifecycle. These ontologies offer a rigorous foundation but remain centred on the scientific experiment and the training process; none addresses the contractual dimension of exchange between autonomous participants.

The work closest to DAIMO is MLDCAT-AP 3.0.0[4] (SEMIC, 2025). MLDCAT-AP extends DCAT to describe AI models as catalogued resources with rich metadata on policy, benchmarks, evaluation, execution, risk, and compute infrastructure. DAIMO treats this representation as the AI-asset layer and concentrates on the layer that MLDCAT-AP leaves open: the act of offering a model across an organisational boundary, the governed deployment of that model as an invocable service, the authorisation of concrete executions under an agreement, and the aggregation of provenance across participants. MLDCAT-AP covers the model description; DAIMO covers the governance bridge that makes the model exchangeable, invocable, auditable, and comparable in a dataspace.

Table 1 makes the MLDCAT-AP relationship explicit. For each MLDCAT-AP construct most likely to co-occur with DAIMO, it records whether the construct is adopted as part of the AI-asset layer, profiled with additional conformance expectations, or complemented by a bridge construct that MLDCAT-AP does not cover. The pattern is consistent: on the AI-asset plane DAIMO preserves the SEMIC vocabulary; on the catalog-dataspace bridge it adds without overriding.

Governing resources and dataspaces

Two threads of work provide the governance substrate that DAIMO composes on top of the model description. The first is the family of W3C governance vocabularies for resources. DCAT 2[5] provides the catalog primitives on which MLDCAT-AP and DAIMO rest. ODRL 2.2[6] offers a consolidated model for permissions, prohibitions, obligations, offers, and binding agreements; these policy constructs are essential for describing governed model offerings and execution authorisations. PROV-O[7] provides the provenance ontology over activities, entities, agents, bundles, and roles, and is the natural substrate for tracing individual executions and their cross-participant aggregation. Pandit and Esteves[18] show how ODRL combines with DPV for health-data scenarios; DAIMO transposes that pattern to the AI domain. These vocabularies are mature and widely adopted but domain-agnostic: none is specific to AI, and none is specific to dataspaces.

The second thread is the emerging set of dataspace standards that describe the exchange itself. Franklin et al.[19] framed dataspaces as a response to persistent heterogeneity, and Otto et al.[20] and Cappiello et al.[21] turn data sovereignty, contracts, and controlled interoperability into explicit design assumptions. The Dataspace Protocol[3] is the W3C-CG specification that defines catalog, contract-negotiation, and transfer-process messages; Eclipse Dataspace Components[2] is

Table 1. Relationship between DAIMO and MLDCAT-AP 3.0.0, class by class. Adopted means the construct remains in the MLDCAT-AP layer. Profiled means DAIMO adds conformance expectations. Bridge extension means DAIMO introduces a construct for the catalog-dataspace relation that MLDCAT-AP does not cover.

MLDCAT-AP class	DAIMO treatment	Notes
it6:MachineLearningModel	Adopted + profiled	AI asset description; DAIMO requires policy, title, and identifier for governed exchange.
it6:Task	Adopted	Target task of a model.
it6:Run	Adopted + profiled	Execution record; DAIMO requires flow, algorithm, agent, timestamp, and authorisation linkage.
it6:Flow	Adopted	Pipeline associated with a run.
it6:Evaluation	Adopted	Measurement result; can be included in cross-participant provenance.
it6:EvaluationMeasure	Adopted	Metric type and value.
it6:Benchmark	Adopted	Benchmark reference used to define comparability conditions.
it6:ComputerInfrastructure	Adopted	Compute environment on which a deployment runs.
it6:Hardware, it6:Library	Adopted	Elements of the compute environment.
it6:HarmRisk	Adopted	Risk annotation on a model.
it6:Modality	Adopted	Input modality.
it6:trainedOn, it6:testedOn	Adopted	Training/test dataset links.
Bridge constructs absent from MLDCAT-AP and added by DAIMO		
(no MLDCAT-AP analogue)	Bridge extension	Governed catalog offering with attached ODRL policy.
(no MLDCAT-AP analogue)	Bridge extension	Execution authorisation grounded in an agreement.
(no MLDCAT-AP analogue)	Bridge extension	Deployment relation between model, infrastructure, and invocable service.
(no MLDCAT-AP analogue)	Bridge extension	I/O formats and authentication conditions.
(no MLDCAT-AP analogue)	Bridge extension	Governable output of an authorised run.
(no MLDCAT-AP analogue)	Bridge extension	Compliance-oriented integrity and audit evidence.
(no MLDCAT-AP analogue)	Bridge extension	Provenance aggregation across participant contexts.
(no MLDCAT-AP analogue)	Bridge extension	Reified comparability conditions for evaluations.
(no MLDCAT-AP analogue)	Bridge extension	Participant role in dataspace exchange.

its reference implementation. One distinction deserves emphasis: EDC is a Java execution framework, not an ontology. The semantics that EDC implements come from DSP (protocol), ODRL (policies), and DCAT (catalog); only a handful of concepts are EDC-specific extensions, and DAIMO uses the participant-context notion only to anchor aggregated provenance to an administrative context. Work on federated ecosystems such as ConSolid[22] further shows that the Semantic Web is evolving towards multi-actor scenarios where governance and interoperability must be designed jointly.

Reusable and executable ontology engineering

A third body of work informs how an ontology profile should be engineered and evaluated. Recent operational ontologies increasingly combine modular design, explicit reuse, public documentation, competency questions, and executable validation artefacts. RePlanIT[23] models digital product passports for ICT equipment and validates the ontology through SHACL and expert-oriented use cases. GloSIS[24] operationalises soil information at global scale through a modular profile aligned with SOSA/SSN and ISO 28258. FATO[25] shows how domain literature, expert input, and competency questions can be combined when a single expert group is insufficient for sector-wide requirements. NutriLink[26] grounds its ontology in a motivating scenario and shows how competency questions become concrete query patterns. ANNO[27] and the SAE J3016 autonomous-driving ontology[28] illustrate the value of explicit ontological commitments when domain terminology is dense and standard-driven. The Explanation Ontology[29] is especially relevant to DAIMO because it treats AI outputs, users, and explanatory artefacts as connected resources and evaluates the model through use cases and competency questions. Woods et al. connect ontology engineering with natural-language processing for industrial maintenance activities[30]. Bareedu et al. derive SHACL validation rules from OPC UA industry standards[31]. Ferranti et al. formalise

Wikidata property constraints through SHACL and SPARQL[32]. Robaldo and Batsakis[33] examine the interplay between SHACL validation and inference on the Time Ontology, and Penteado et al.[34] study methodologies for publishing linked open government data. Taken together, these works indicate that a practically useful ontology should not only define classes and properties. It should make the full chain from requirements, reuse decisions, alignments, validation constraints, examples, documentation, and reproducibility conditions inspectable.

Methodology and requirements

DAIMO was developed with the Linked Open Terms (LOT) methodology[35] because the contribution is not a stand-alone upper ontology but an integration profile whose adequacy depends on traceable requirements, disciplined reuse, and reproducible validation. We instantiated LOT in five activities: define the operational setting, elicit competency questions from that setting and from relevant specifications[36], map each question to reusable vocabularies where possible, introduce bridge constructs only for unmet requirements, and validate the resulting artefact through reasoning, SHACL constraints, negative tests, and executable queries. The subsections below report these activities in the order in which they shaped the ontology.

Illustrative scenario

The illustrative scenario is used as a requirements instrument rather than as evidence of field deployment. It abstracts a recurrent dataspace situation in which five anonymised actor types exchange a geospatial risk-prediction model: Provider A (research organisation publishing the model), Consumer B (municipal consumer invoking predictions), Platform C (operator of the federated dataspace), Evaluator D (scientific evaluator publishing comparative measurements), and Compliance E (governance body auditing the chain). Together these actors expose the lifecycle tasks a catalogue-dataspace bridge must support: publishing a model as a governed asset, discovering it under policy and quality constraints, invoking a deployment, tracing execution evidence across participants, and comparing evaluation results under reproducible conditions. The evaluation section later instantiates the same scenario to make the requirements and validation artefacts executable.

Requirements specification

The requirements were derived from three complementary sources: the illustrative scenario, the public specification of MLDCAT-AP 3.0.0, and the public documentation of Eclipse Dataspace Components and the Dataspace Protocol. The resulting twenty-three competency questions are grouped by actor and lifecycle stage. Category R (registration and publication, five questions) concerns the provider's need to publish a model as a governed asset with identity, licence, policy, and invocation contract. Category D (discovery and selection, four questions) captures the consumer's need to locate a suitable model by task, domain, policy, and quality threshold. Category E (execution and auditability, five questions) concerns invocation, operational traces, and evidence reconstruction. Category V (evaluation and reproducibility, five questions) covers comparisons under homogeneous evaluation conditions. Category G (governance bridge, four questions) isolates cross-actor questions that cannot be answered by the reused vocabularies without additional bridge semantics. Table 3 summarises the actor-category mapping; Table 4 traces each requirement family to the modelling commitments that answer it; the full set of twenty-three questions in natural language appears in the competency-question appendix.

The question set also fixes the intended reasoning profile. Eleven of the twenty-three questions require explicit reasoning, either through subclass paths, subproperty chains, or SPARQL aggregation; the remaining twelve are answerable without inferencing. This split is important for an integration ontology. It shows where formal semantics add interoperability value while keeping the ordinary discovery and reporting questions executable on standard SPARQL endpoints.

The non-functional requirements constrain both modelling content and publication quality. The artefact is constrained to OWL 2 DL for decidability and HermiT compatibility, is released under CC-BY 4.0 (Apache 2.0 for validation code), and uses persistent URIs under https://w3id.org/pionera/daimo with content negotiation, WIDOCO documentation[37], and accessible SHACL validation. A reuse-by-default rule binds the modelling work: each DAIMO term must justify why none of DCAT, DCAT-AP, MLDCAT-AP, ODRL, PROV-O, DSP, or the EDC extension already covers the required meaning.

Core design decisions

Three structural decisions shape the rest of the design.

Decision 1: reuse preferentially. DAIMO preserves the conceptualisations of established vocabularies whenever they already capture the intended meaning. MLDCAT-AP remains responsible for the model, task, run, pipeline, evaluation, and compute-environment layer; DCAT remains responsible for cataloguing and service description; and ODRL remains responsible for policy offers and agreements. DAIMO intervenes only where these layers must be connected for governed exchange.

Introducing DAIMO-specific equivalents for any of these would violate the principle of minimal ontological commitment[38] and fracture the DCAT-AP ecosystem without adding expressive power. A new class is therefore justified only when the union of the reused vocabularies leaves a concrete gap that prevents answering at least one competency question.

Decision 2: separate three complementary planes. The architecture distinguishes three planes that DAIMO connects but does not replace. The dataspace exchange plane covers participant contexts, catalogue exchange, and protocol-level interaction.

Table 2. Reuse matrix: families of related work against the five operational attributes that a profile for governed AI models must cover. • = primary coverage; ○ = partial coverage; — = not covered.

Family / artefact	Publication	Policy	Invocation	Traceability	Evaluation
Model cards, factsheets, datasheets[8–10]	○	—	—	—	○
ML-Schema, MEX, OntoDM, DMOP[12–15]	—	—	—	○	•
MLDCAT-AP 3.0.0[4]	•	○	—	•	•
DCAT 2[5]	•	—	○	—	—
ODRL 2.2[6]	—	•	—	—	—
PROV-O[7]	—	—	—	•	○
DSP / EDC[2,3]	○	○	•	—	—
DAIMO (this work)	•	•	•	•	•

Table 3. Actors involved in the illustrative scenario and the competency-question categories derived from them.

Actor	Category	Need	CQs
Model provider	R	Publish a model as a governed asset with identity, licence, policy, and invocation contract	CQ-R1–CQ-R5
Model consumer	D	Discover models by task, domain, policy, and quality threshold	CQ-D1–CQ-D4
Platform operator	E	Invoke services, trace executions, and reconstruct operational evidence	CQ-E1–CQ-E5
Evaluator	V	Compare results under a shared context and inspect reproducibility	CQ-V1–CQ-V5
Governance (multi-actor)	G	Bridge offer, deployment, authorisation, and cross-participant provenance	CQ-G1–CQ-G4

Table 4. Traceability from operational requirement to modelling commitment. The table is intended to make visible why each modelling group exists and which evaluation artefact exercises it.

Requirement family	Question the profile must answer	Primary DAIMO commitment	Evidence
Publication	Is this model a governed asset in a particular catalog context?	Governed offering record linking model, publisher, catalog context, and applicable policy.	CQs R1–R5, G1
Discovery	Can a consumer filter models by task, domain, policy, and evaluation threshold?	MLDCAT-AP model/task/evaluation semantics plus policy links and evaluation context.	CQs D1–D4
Invocation	Which service, authentication method, and I/O contract apply to a selected model?	Deployment description plus service and I/O-contract commitments, allowing several endpoints per hosted model.	CQs E1, G2
Authorisation	Which agreement authorised a concrete execution by a concrete participant?	Negotiated agreement linked to the participant role and execution it covers.	CQs E2, G3; INV-2, INV-4
Auditability	Can a governance actor reconstruct artefacts, evidence, and provenance across contexts?	Derived artefact, evidence record, and cross-participant provenance bundle.	CQs E4, E5, G4; INV-1
Evaluation	Are measurements comparable under the same task, dataset version, protocol, and seed?	Shared evaluation context capturing task, dataset version, protocol, and seed.	CQs V1–V5

The governance-operation plane covers offers, agreements, authorisations, and evidence records that make exchange auditable. The AI-asset plane covers model descriptions, tasks, runs, evaluations, and compute environments. DAIMO binds these planes at their lifecycle boundaries instead of redefining the vocabularies that own them.

Decision 3: one class per gap, not per theme. Given reuse preference, each class introduced by DAIMO must justify its existence by a concrete gap in the union of the reused vocabularies and by at least one competency question that cannot be answered without it. Strict application of this criterion produces nine top-level classes (AIAssetOffering, ParticipantRole, ModelDeployment, IOContract, ExecutionAuthorization, DerivedArtifact, CrossParticipantProvenanceRecord, AuditEvidence, and SharedEvaluationContext) plus five role subclasses. Any candidate class failing both conditions simultaneously is discarded.

The DAIMO ontology

Modular architecture and metrics

The ontology specification follows directly from the requirements in Section . The competency questions delimit five tasks: publication as a catalogue record, policy-and-quality-sensitive discovery, invocation through typed services, traceability of individual executions, and contextualised evaluation. Responsibilities outside this boundary, such as detailed training pipelines, fine-grained privacy-policy semantics, or formal derivation theory, already belong to mature vocabularies. DAIMO therefore contributes the boundary constructs needed to make those vocabularies interoperate when an AI model circulates between autonomous participants.

The implementation is decomposed into three Turtle modules, each serving a distinct adoption need. The core module (daimo-core.ttl) contains the fourteen DAIMO classes, twenty-nine object properties, eight datatype properties, the global disjointness axioms, and the property characterisations (functional, asymmetric, inverse) that capture TBox-level type invariants. The alignment module (alignment.ttl) encapsulates the seven rdfs:subClassOf and six rdfs:subPropertyOf declarations that connect DAIMO to reused vocabularies, plus minimal declarations of external terms with rdfs:seeAlso pointers to their source specifications. The SHACL module (daimo-shapes.ttl) contributes eighteen sh:NodeShapes: nine structural shapes, three conformance shapes over reused classes, and six cross-class invariants expressed with sh:SPARQLConstraint. Because the SHACL module is declared as an independent owl:Ontology with its own owl:versionIRI, the validation layer can evolve without forcing a new logical vocabulary release.

Table 5 summarises the size and expressivity of the artefact, computed from the ontology files and validation reports. The numbers let a reader compare DAIMO with previously published ontologies in the same family.

[TODO: A complementary OntoMetrics-style table (Protégé plugin) is missing, breaking axiom counts down by type: SubClassOf, EquivalentClasses, DisjointClasses, ObjectPropertyDomain, ObjectPropertyRange, FunctionalObjectProperty, AsymmetricObjectProperty, InverseObjectProperties, SubDataPropertyOf, DataPropertyDomain, DataPropertyRange, AnnotationAssertion, etc. RePlanIT §5.2 and most ontology-journal papers report this table; it gives the reviewer a quantitative measure of axiomatic richness independent of class and property counts.]

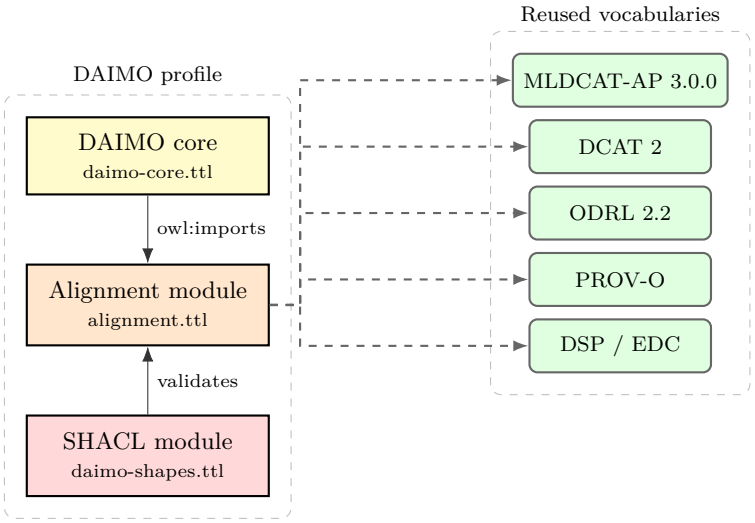

Figure 1. Modular architecture of DAIMO and reused vocabularies. Yellow, orange, and red boxes denote the three DAIMO modules (core, alignment, shapes); green boxes denote the reused vocabularies. Dashed arrows from the alignment module carry the rdfs:subClassOf and rdfs:subPropertyOf declarations. The core module imports alignments for inference; the SHACL module validates data against the combined TBox.

Table 5. DAIMO artefact metrics verified against package files.

Dimension	Value
Declared logical profile	OWL 2 DL
DAIMO top-level classes	9
DAIMO subclasses (ParticipantRole)	5
Total DAIMO-specific classes	14
Object properties (owl:ObjectProperty)	29
Datatype properties (owl:DatatypeProperty)	8
Functional properties (owl:FunctionalProperty)	28
Asymmetric properties (owl:AsymmetricProperty)	5
Explicit inverse pairs (owl:inverseOf)	5
rdfs:subClassOf to externals	7
rdfs:subPropertyOf to externals	6
Named disjointness axiom	1
Total sh:NodeShapes	18
completeness (one per DAIMO class)	9
conformance over reused classes	3
cross-class SPARQL invariants	6
Competency questions	23
OOPS! pitfalls (Critical/Important/Minor)	0 / 0 / 2

Classes and alignments

The class layer is organised around lifecycle boundary objects rather than around broad AI-governance themes. The nine top-level classes, together with the five ParticipantRole subclasses, are the minimal set required by the competency questions under the reuse rule introduced above. Table 6 summarises each class, its axiomatic alignment to reused vocabularies, and the competency questions it supports. Figure 2 gives the corresponding structural view.

The absence of an external alignment is also a modelling decision. IOContract is not a dcat:DataService: a service is an accessible resource or endpoint, whereas

Table 6. DAIMO classes, axiomatic alignments, and the competency questions they answer.

Class	Alignment	Role	CQs
daimo:AIAssetOffering	dcat:CatalogRecord	Catalog record that publishes a model as a governed offer in a dataspace; carries an ODRL policy via daimo:hasOfferPolicy.	R1, R2, R5, G1
daimo:ParticipantRole (+5 non-disjoint subclasses)	prov:Role	Role a participant plays with respect to an AI asset within a dataspace context.	R5
daimo:ModelDeployment	prov:Entity	Running or hosted instance of a model, exposed as a dcat:DataService over an it6:ComputerInfrastructure.	E1, E3, G2
daimo:IOContract	(no external alignment)	Minimal actionable contract: I/O formats, authentication method, optional schemas.	R4, E1, G2
daimo:ExecutionAuthorization	odrl:Agreement	ODRL agreement derived from a DSP negotiation that authorises concrete executions via daimo:authorizesRun.	E2, E5, G3
daimo:DerivedArtifact	prov:Entity, dcat:Resource	Governed and catalogable output produced by an execution under authorisation.	E5, G4
daimo:CrossParticipantProvenanceRecord	prov:Bundle	Aggregation of activities and entities across participant contexts forming an audit narrative.	G4
daimo:AuditEvidence	prov:Entity	Compliance-oriented evidence record with structured spdx:Checksum, signer, and timestamp.	E4
daimo:SharedEvaluationContext	(no external alignment)	Reified comparability conditions: task, dataset, version, protocol, and random seed.	V1, V2, V3

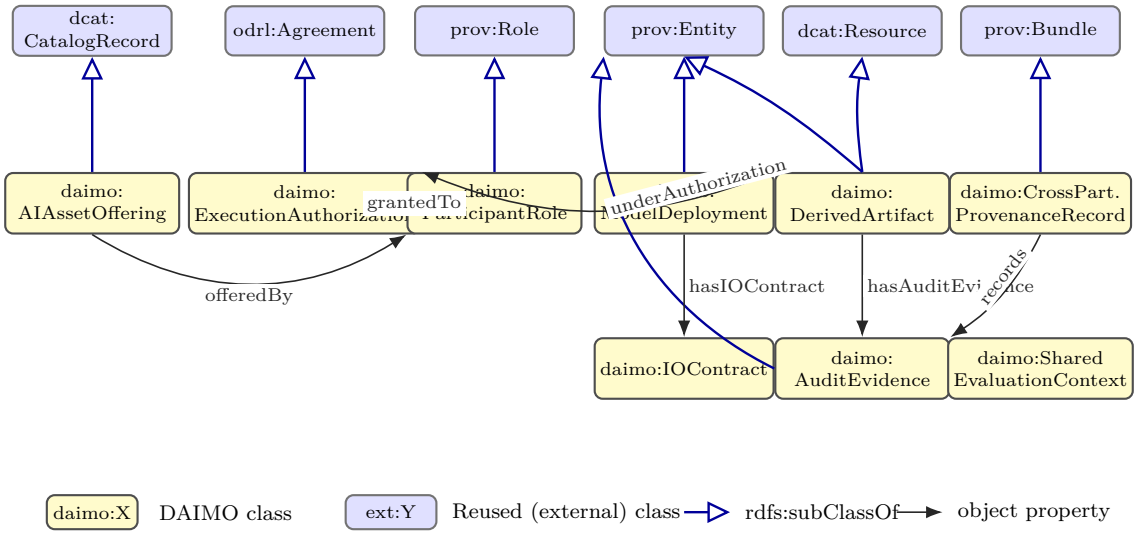

Figure 2. Overview of the DAIMO classes (yellow) with their axiomatic alignments to the reused vocabularies (blue) via rdfs:subClassOf (open-triangle arrows), and the main profile properties that articulate the profile (labelled object-property arrows). DerivedArtifact has two external parents, prov:Entity and dcat:Resource; AuditEvidence is placed in the bottom row but is also a prov:Entity. Classes IOContract and SharedEvaluationContext carry no external alignment by design. Properties whose targets are classes of the reused vocabularies (offersModel to it6:MachineLearningModel, authorizesRun to it6:Run) are declared in Table 6 and are omitted here for readability.

the contract describes syntactic and authentication expectations over an exchange. Nor is it a dct:Standard, because the contract enumerates concrete invocation obligations rather than referring to an external specification. SharedEvaluationContext is likewise not an ML task. It reifies the combination under which two evaluations are comparable, rather than the abstract task those evaluations instantiate. Assigning either class a generic external parent would add little semantic value while increasing the chance of unintended entailments.

In the illustrative scenario, the provider publishes three governed model offerings in a federated catalog. Each offering records which model is made available, which participant publishes it, which policy applies, and in which catalog context the offer is valid. The model selected by the municipal consumer is deployed as a hosted service with REST and gRPC endpoint descriptions, each carrying its own invocation contract. After the dataspace negotiation, the consumer obtains an authorisation associated with the executions it performs. Each execution produces a prediction artefact,

and the chain is completed by audit evidence and a cross-participant provenance record spanning provider, consumer, and platform contexts.

Registration and participation. AIAssetOffering and ParticipantRole jointly solve the publication-side gap. MLDCAT-AP describes the model, and DCAT records catalog metadata, but neither expresses the governed act of making a model available to another dataspace participant. The offering is therefore treated as the catalog-facing record of a model in a specific dataspace, scoped by policy, publisher, target model, and participant context. ParticipantRole, with five non-disjoint specialisations, records the role a legal agent plays in relation to an asset and a dataspace context. The two classes jointly answer CQ-R1, CQ-R2, CQ-R5, and CQ-G1.

Execution plane. ModelDeployment and IOContract cover the invocation-side gap. DCAT supplies catalog-visible services, but it does not say which model a service hosts or which syntactic and authentication expectations a consumer must satisfy to call it. A deployment represents the running or hosted instance of a model on compute infrastructure. It may be exposed through one or several services, and each service may declare an invocation contract covering input/output format, authentication method, and optional schema references. This group answers CQ-E1, CQ-E3, and CQ-G2.

Authorisation and derivation. ExecutionAuthorization and DerivedArtifact cover the authorisation-to-output chain. ODRL supplies binding agreements, and PROV-O supplies general generation and derivation relations, but neither records the specific act of authorising a concrete machine-learning run or the governance-visible artefact that the run produces. The authorisation is modelled as the negotiated agreement that binds permissions to executions; the derived artefact is modelled as the governed output of such an execution. INV-1 enforces the consistency of this chain. The group answers CQ-E2, CQ-E5, CQ-G3, and CQ-G4.

Audit and provenance aggregation. AuditEvidence and CrossParticipantProvenanceRecord cover the auditability-across-boundaries gap. SPDX supplies checksums, and PROV-O supplies provenance bundles, but neither addresses compliance-oriented evidence that spans administrative contexts. Audit evidence combines an integrity value, a signer, and a timestamp; the cross-participant provenance record aggregates the relevant activities and entities across participant contexts, producing an audit narrative that a governance actor can traverse end-to-end. The group answers CQ-E4, CQ-G4.

Evaluation context. SharedEvaluationContext covers the comparability gap that plain MLDCAT-AP evaluations leave open. MLDCAT-AP records a measurement attached to a model; it does not reify the combination of task, dataset version, evaluation protocol, and random seed that makes two measurements comparable. Two evaluations sharing that context are comparable by construction; evaluations that differ on any coordinate are not. The class answers CQ-V1, CQ-V2, CQ-V3.

Justifying the principal alignments. Two of the seven rdfs:subClassOf alignments carry more semantic commitment than the others. Plausible alternative modelling choices exist for both, so their justification is made explicit.

AIAssetOffering $\sqsubseteq$ dcat:CatalogRecord, not dcat:Dataset. DCAT carefully distinguishes the offered resource (the dcat:Dataset, the thing of interest) from the record that describes that resource in a concrete catalog (the dcat:CatalogRecord, the metadata artefact). We align AIAssetOffering to dcat:CatalogRecord because an offering is conceptually the cataloguing record of the model in a specific dataspace, with a policy and a set of metadata specific to that context. The offering is not the model itself, whose AI-asset description remains in the MLDCAT-AP layer. A counter-reading that subsumed AIAssetOffering under dcat:Dataset would conflate two distinct entities: the model (which can appear in several catalogs with different conditions) and its offering in one specific catalog (which is one and has its own policy). This distinction is operationally relevant: the same model may appear in several offerings with distinct policies, and the property daimo:offersModel, aligned to foaf:primaryTopic, captures that the offering is about the model without being identified with it.

ExecutionAuthorization $\sqsubseteq$ odrl:Agreement, not prov:wasDerivedFrom odrl:Agreement. An execution authorisation is the specific result of a DSP negotiation that binds concrete ODRL permissions to concrete invocations, and ODRL models this binding instrument as odrl:Agreement: the equivalent of a contract between an assigner and an assignee, carrying effective permissions and restrictions. The alternative of relating an authorisation to an agreement via prov:wasDerivedFrom would treat the authorisation as a distinct artefact from the agreement and introduce a distinction ODRL does not require. Subsumption preserves the existing ODRL semantics for permissions, prohibitions, and obligations, while allowing existing ODRL-aware tools to recognise DAIMO authorisations as agreements. DAIMO adds only the bridge relation daimo:authorizesRun, which links the agreement to the it6:Runs it covers: a relation ODRL does not prescribe but the bridge to the provenance plane requires.

The remaining alignments are justified below, ordered from the most to the least contentious.

ParticipantRole $\sqsubseteq$ prov:Role. PROV-O scopes Role to a qualified association inside a single activity, while the DAIMO role is scoped to a dataspace context and typically outlives any single execution. The subsumption is therefore defensible only in the is-a-kind-of sense: a DAIMO role is a role in the PROV sense whenever a specific activity qualifies it, and the scope difference is captured by the additional property daimo:inContext linking the role to an edc:ParticipantContext. A less committal alternative would be skos:related to prov:Role, which we rejected because it would prevent

reasoners from classifying DAIMO roles as PROV roles when they do qualify an association. We accept the tighter subsumption with the understanding that contexts extending beyond a single activity are an additional constraint, not a contradiction.

ModelDeployment $\sqsubseteq$ prov:Entity. A deployment is a persistent artefact over which activities happen (model loading, inference, logging), not an activity itself. Modelling it as prov:Activity would force each deployment to carry startedAtTime/endedAtTime, which fits a run but not a long-lived hosted instance. Aligning it to prov:Entity lets the deployment-model relation act as a genuine entity-to-entity link, and preserves the intuition that running a model on infrastructure produces a new governable entity rather than a new activity.

DerivedArtifact $\sqsubseteq$ prov:Entity, dcat:Resource. The double subsumption reflects two complementary roles: a derived artefact is a provenance entity produced by a run, and simultaneously a catalogable resource that a governance actor may list, describe, and distribute. prov:Entity and dcat:Resource are compatible: DCAT itself treats dcat:Resource as the top of its resource hierarchy with no commitment against PROV. The OWL-RL closure over the demonstrator confirms that no unintended typing emerges from the double parent: the entailment-verification check inspects every inferred superclass of DerivedArtifact and raises zero warnings. The double alignment is what allows a single artefact to be cited both from a provenance bundle (as prov:Entity) and from a DCAT catalog record (as dcat:Resource).

CrossParticipantProvenanceRecord $\sqsubseteq$ prov:Bundle. PROV-O offers two collection constructs: prov:Bundle (a named set of provenance descriptions with its own attribution) and prov:Collection (a set of entities with no attached provenance meta-semantics). The record is exactly the first: an attributable named grouping of provenance descriptions across several edc:ParticipantContexts. Aligning to prov:Collection would lose the meta-provenance layer that makes audit narratives inspectable (a bundle itself has a wasAttributedTo agent and a generatedAtTime); aligning to both would introduce redundancy. prov:Bundle is the right choice.

AuditEvidence $\sqsubseteq$ prov:Entity. Evidence is a produced artefact (a hash value plus its signer and timestamp), not the act of producing it. The auditing activity is a separate prov:Activity instance associated with the compliance officer; AuditEvidence is what that activity generates. Aligning the class to prov:Activity would conflate the two and break INV-4-like invariants that compare authorisation validity against evidence generation time. The choice of prov:Entity keeps the activity-entity distinction that PROV relies on.

Axiomatic foundations

A taxonomy alone does not pin down meaning; it only names the types. DAIMO therefore combines the class hierarchy with an axiomatic infrastructure that fixes what each class is, what it is not, and how conforming instances must behave. Three complementary strata make up this infrastructure: OWL axioms over classes and properties, SHACL nodal constraints over instances, and SHACL-SPARQL cross-class invariants. The OWL stratum is described here; the SHACL strata are reported in Section as part of validation.

On the OWL stratum, disjointness between the nine DAIMO top-level kinds is declared through the named axiom daimo:TopLevelKindsDisjointness, an instance of owl:AllDisjointClasses. The five ParticipantRole subclasses are deliberately excluded: one legal agent may hold several roles at once (provider and operator, for instance), which reflects that roles are context-dependent rather than mutually exclusive.

The disjointness core is complemented by property characterisations. Twenty-eight properties are declared functional where the corresponding structural shape also imposes maximum cardinality, so cardinality is represented in both the logical and validation layers. Five lifecycle relations are asymmetric, which formally captures that a deployment is not the model it deploys, an authorisation does not coincide with the execution it authorises, and so on. Five inverse pairs support bidirectional traversal without forcing the publisher of the graph to materialise both directions.

The SHACL stratum splits into two families. Each DAIMO governance class carries a structural nodal shape that prescribes which properties are mandatory in a well-formed instance; these nine shapes are the syntactic guarantee that a publisher has supplied everything the semantic invariants need. Three conformance shapes (OfferInDAIMOShape, MachineLearningModelInDAIMOShape, and RunInDAIMOShape) establish which properties of reused classes are mandatory when a graph declares adherence to the DAIMO profile. The distinction between the two families is intentional: DAIMO does not override the underlying ontology, because that would fracture the ecosystem. Instead, it declares a commitment to a concrete subset of properties on the external classes, making the adherence criterion explicit without modifying the original definitions.

Above the shapes sit the six SHACL-SPARQL cross-class invariants (INV-1 through INV-6), examined in detail during validation. They encode rules whose meaning cannot be expressed as a local constraint on a single shape because the rules span several classes simultaneously. Listing 1 shows how the OWL axioms are encoded for the concrete case of ExecutionAuthorization: the subclass of odrl:Agreement, the asymmetry of authorizesRun, the functionality of its inverse, and membership in the named disjointness axiom.

```
daimo:ExecutionAuthorization a owl:Class ;
    rdfs:subClassOf odrl:Agreement ;
    skos:definition "An ODRL agreement produced by a
        DSP contract
        negotiation that binds specific permissions to run
            invocations..."@en .
```

```
daimo:authorizesRun a owl:ObjectProperty , owl:
      AsymmetricProperty ;
    rdfs:domain daimo:ExecutionAuthorization ;
    rdfs:range it6:Run .

daimo:authorizedBy a owl:ObjectProperty , owl:
      FunctionalProperty ;
    owl:inverseOf daimo:authorizesRun ;
    rdfs:domain it6:Run ;
    rdfs:range daimo:ExecutionAuthorization .

daimo:TopLevelKindsDisjointness a owl:AllDisjointClasses
      ;
    owl:members ( daimo:AIAssetOffering
                  daimo:ExecutionAuthorization
                  daimo:ModelDeployment
                  daimo:DerivedArtifact
                  daimo:IOContract
                  daimo:AuditEvidence
                  daimo:
                        CrossParticipantProvenanceRecord
                  daimo:SharedEvaluationContext
                  daimo:ParticipantRole ) .
```

Listing 1: Core axioms around ExecutionAuthorization.

Reuse surface. Table 7 summarises the reused vocabularies and their concrete use. Three observations matter. First, the alignments to DCAT and ODRL target standard vocabularies rather than implementation-internal entities. Second, DAIMO maintains informative mappings to DSP through skos:closeMatch and skos:related, not subsumption: DSP is a messaging protocol rather than an asset ontology, and subsuming DAIMO classes under DSP constructs would introduce spurious typings. Third, of the EDC extension only edc:ParticipantContext appears explicitly, where it anchors aggregated provenance to an administrative context.

Three subproperties that might seem natural (authorizesRun $\sqsubseteq$ prov:used; grantedTo $\sqsubseteq$ prov:qualifiedAssociation; evidenceOf $\sqsubseteq$ prov:hadActivity) are deliberately not declared, because each would produce unintended typings on authorisations, agents, or evidence under RDFS closure. The entailment-verification check reported below catches errors of this kind.

Evaluation

Evaluation is organised around four validation claims rather than around an implementation test log. Following the usual distinction between requirements coverage, formal correctness, constraint-based validation, and application-level usefulness[43], we ask whether the profile can represent the target exchange, whether the OWL artefact is formally coherent, whether the SHACL constraints discriminate valid from invalid graphs, and whether the competency questions are executable over the materialised graph. This evidence pattern follows recent ontology papers that combine scenario grounding, formal artefact checks, and CQ or use-case validation, including RePlanIT[23], GloSIS[24], FATO[25], and the Explanation Ontology[29]. The complete reproducibility package accompanies the artefact.

Illustrative demonstrator

The requirements scenario is instantiated as a controlled reproducibility demonstrator. The resulting graph contains 225 triples, exercises every DAIMO class at least once, and provides the data over which the twenty-three competency questions are evaluated. All organisations, model names, dataset versions, and metric values are anonymised or synthetic. The demonstrator is not a deployment study; it is a compact instantiation used to test representational adequacy and to make the validation artefacts executable.

Provider A publishes three governed model offerings in a federated catalogue: two versions of the same geospatial risk-prediction model and one read-only variant with a stricter policy. The offerings share task and domain descriptions but differ in the permissions attached to them. This part of the scenario exercises the distinction between a model as an AI asset and an offering as a contextual catalogue record with its own usage conditions.

The selected model is deployed on a described compute environment and exposed through two service interfaces: a REST endpoint with JSON input and bearer-token authentication, and a gRPC endpoint with protobuf and mutual TLS. The multi-endpoint usage exercises the modelling decision that a single deployment may expose several services and contracts, rather than forcing one endpoint per deployment.

Consumer B obtains an execution authorisation with explicit validity dates, invokes the model, and receives a derived prediction artefact. The execution is linked to audit evidence containing an integrity record, signer, and timestamp, so that a governance actor can reconstruct how the artefact was produced and under which authorisation. A cross-participant provenance record aggregates the chain across provider, consumer, and platform contexts. Evaluator D publishes two synthetic evaluation results under the same benchmark, protocol, and random seed, allowing the two model versions to be compared under shared conditions. Figure 3 illustrates the instantiated scenario.

Formal verification

HermiT, invoked through owlready2, confirms that the union of the core, the alignments, and the example graph is logically consistent with no unsatisfiable classes; reasoning takes 0.72 s. OWL-RL materialisation (the owlrl library) derives 1 171 additional triples on top of 817 initial ones (1 988 in total) in 0.27 s, with no individuals typed as owl:Nothing and no unsatisfiable subclasses.

Logical consistency is necessary but not sufficient. An ontology may be consistent and still infer, for instance, that audit evidence is a provenance activity rather than an entity produced by an activity. We therefore added an entailment-verification check over the OWL-RL closure: for each DAIMO class, the check enumerates every

Table 7. Vocabularies reused by DAIMO and the concrete use of each.

Vocabulary	Reused terms	Use in DAIMO
MLDCAT-AP 3.0.0 (it6:)	MachineLearningModel, Task, Run, Flow, Evaluation, EvaluationMeasure, Benchmark, ComputerInfrastructure, Hardware, Library, HarmRisk, Modality, trainedOn, testedOn	AI-asset plane; DAIMO does not redefine the model.
DCAT 2 (dcat:)	Catalog, Dataset, Distribution, DataService, CatalogRecord, endpointURL	Publication and discovery.
ODRL 2.2 (odrl:)	Offer, Agreement, Permission, Prohibition, hasPolicy, target, assigner, assignee	Policy, offer, and agreement.
PROV-O (prov:)	Activity, Entity, Agent, Bundle, Role, wasGeneratedBy, wasAssociatedWith	Provenance and roles.
FOAF (foaf:)[39]	primaryTopic	Subproperty target for linking an offering record to the model it is about.
SPDX (spdx:)[40]	Checksum, algorithm, checksumValue	Audit-evidence integrity.
SKOS (skos:)[41]	definition, example, closeMatch, related	Term documentation and non-subsumptive mappings.
DSP (dspace:)	ContractOffer, ContractNegotiation, TransferProcess	Informative skos:closeMatch/skos:related mappings; not subsumption.
EDC (edc:)	ParticipantContext	Only EDC-specific extension referenced.

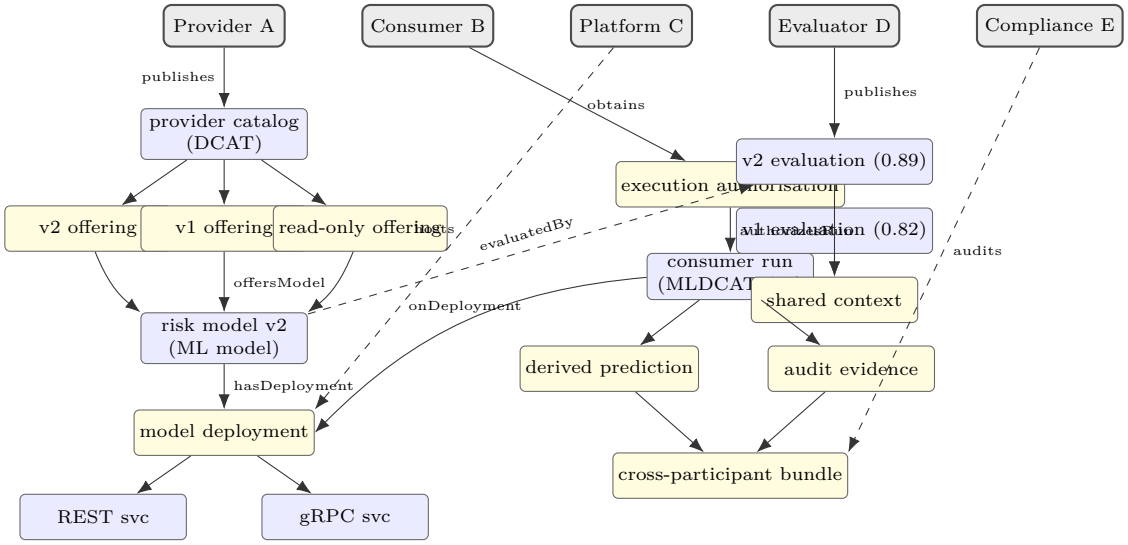

Figure 3. Illustrative demonstrator used for validation with five anonymised participants. Grey rounded boxes on the top row are the participant contexts; yellow-shaded nodes are governance constructs introduced by DAIMO; light-blue nodes are resources described with established vocabularies. Solid arrows denote domain relations; dashed arrows show the participant role associated with an instance.

inferred superclass and tests it against a list of forbidden ones. The report covers the fourteen DAIMO classes (nine top-level plus five role subclasses) and raises no warning. This check complements consistency reasoning by detecting modelling-inappropriate alignments that remain logically satisfiable.

The OOPS! scanner[45], queried through its REST API on the union of the core and alignments, reports zero Critical pitfalls, zero Important ones, and two Minor ones: P13 ("inverse relationships not declared") on 34 elements with no semantically meaningful inverse, and P04 ("isolated ontology elements") on seven locally declared external classes. Both arise from the reuse-by-default discipline the profile applies.

The final layer of formal verification targets the alignment module itself. The subclass and subproperty bridges connect DAIMO to its five parent vocabularies: DCAT, MLDCAT-AP, ODRL, PROV-O, and FOAF[39]. The entailment-verification check inspects the OWL-RL closure of each alignment and confirms that none produces a semantically incorrect superclass on any DAIMO governance class, raising zero warnings over the fourteen classes. Three tempting PROV-O subproperty bridges are deliberately excluded from the module for the same reason: each would have produced a

forbidden typing under RDFS closure, as discussed in the axiomatic-foundations subsection.

SHACL validation: structural, invariants, and negative test bench

The SHACL layer contains three shape families, each answering a different validation question. Structural shapes ask whether every DAIMO-specific instance carries the mandatory properties of its class. Conformance shapes ask whether reused external classes satisfy the additional obligations that the profile imposes on them. SPARQL invariants ask whether relations across several classes remain coherent. Together, the three families move validation from local completeness to graph-level consistency: from "is this instance well-formed?" to "is this graph internally coherent?".

The nine structural shapes impose minimum completeness per DAIMO class, and the 225-triple positive graph conforms to them. The three conformance shapes add operational obligations to reused constructs: governed offers must identify their assigner and target; machine-learning models must carry policy, title, and identifier information; and execution records must include flow, algorithm, associated agent, and timestamp. Additional constraints restrict authentication methods to a closed vocabulary and evaluation protocols to recognised protocol-name patterns.

Above this structural stratum sit the six SHACL-SPARQL cross-class invariants. INV-1 guarantees consistency of the derivation-authorisation chain: every derived artefact must point to an authorisation that covers precisely the execution from which the artefact was derived. INV-2 requires the actor recorded in provenance for an execution to match the beneficiary in at least one covering authorisation. INV-3 checks data-plane coherence: a service exposed by a deployment must serve the same model the deployment declares, ruling out an endpoint that advertises one model and serves another. INV-4 ensures that an authorisation cannot justify an execution that started after the authorisation lapsed. INV-5 requires the policy attached to an offering to govern the model named in that offering. INV-6 closes the same triangle on the provider side by checking consistency between the catalogue-record attribution and the policy assigner. The positive graph conforms to all six invariants without warnings.

Listing 2 shows the concrete encoding of INV-5 as sh:SPARQLConstraint, illustrating the FILTER NOT EXISTS pattern that captures the disjunction between policy-level and per-permission targets.

```
daimo:OfferingPolicyTargetInvariant a sh:NodeShape ;
    sh:targetClass daimo:AIAssetOffering ;
    sh:sparql [
        a sh:SPARQLConstraint ;
        sh:message "INV-5 violation: offering's offersModel
              does not
                    appear as odrl:target of the attached
                          policy."@en ;
        sh:prefixes daimo:_invariantPrefixes ;
        sh:select """
            SELECT $this ?model ?policy WHERE {
                $this daimo:offersModel ?model ;
                      daimo:hasOfferPolicy ?policy .
                FILTER NOT EXISTS { ?policy odrl:
                     target ?model }
                FILTER NOT EXISTS { ?policy odrl:
                     permission/odrl:target ?model }
            }
        """ ;
    ] .
```

Listing 2: INV-5 encoded as a sh:SPARQLConstraint.

Shapes that conform on the positive graph are necessary but not sufficient to show that constraints are semantically discriminating. A trivially satisfiable constraint on any graph would pass positive validation without yielding real guarantees. For this reason DAIMO also includes a companion negative graph with six deliberately invalid cases, each crafted to violate exactly one invariant. The validation workflow verifies two conditions: global validation fails, and every expected focus node appears in the report with the specific invariant's message. All six invariants fire correctly on their designated violation. This evidence complements positive validation and grounds the claim that the invariants capture operational rules.

Competency questions

The twenty-three competency questions are expressed as SPARQL queries and run over the OWL-RL materialised closure of the positive graph. Table 8 summarises coverage by category, observed row counts, and queries requiring explicit reasoning; the full set of queries appears in the SPARQL-query appendix. All twenty-three return at least one row, confirming that the demonstration graph contains the information needed to answer every question with the vocabulary, alignments, and constraints specified by the profile.

Two observations deserve emphasis. CQ-R2 and CQ-E1 depend on a subproperty alignment between the offering–model relation and foaf:primaryTopic; without reasoning they return no rows on a plain SPARQL engine. The dependency is documented rather than defective: the alignment makes offerings discoverable through a standard descriptive relation, a benefit that materialises only under inference. A consumer who must run those queries on a plain endpoint can materialise the closure beforehand or rewrite the queries to enumerate both properties. The second observation is that CQ-G2 returns two rows, one per endpoint of the multi-protocol REST and gRPC deployment. This confirms that modelling several services per deployment is visible in query results.

Two fragments illustrate the pattern. Listing 3 shows CQ-G3 ("which authorisation authorised a specific run, and which ODRL agreement backs it?"): the query traverses the authorisation link from an execution to its agreement context, then lifts the agreement's assigner and assignee. Listing 4 shows CQ-V3 ("ranking of models under the same shared evaluation context and metric"): the query joins evaluations against a fixed context, orders by metric value, and binds the model

Table 8. Coverage of the 23 competency questions. Each CQ returns the indicated row count over the positive graph under OWL-RL closure. Boldface marks queries that require explicit reasoning beyond plain SPARQL.

Category	Queries (rows)	N	Inference required
Registration and publication (R)	R1(3) R2(1) R3(1) R4(2) R5(2)	5	R2 (subPropertyOf chain), R5 (subClassOf path)
Discovery and selection (D)	D1(3) D2(2) D3(4) D4(1)	4	D1 (task subtype), D2 (policy matching)
Execution and auditability (E)	E1(4) E2(1) E3(2) E4(1) E5(1)	5	E1 (offersModel $\sqsubseteq$ foaf:primaryTopic), E2 (agreement-run union)
Evaluation and reproducibility (V)	V1(1) V2(1) V3(2) V4(1) V5(4)	5	V2, V3 (aggregation)
Governance bridge (G)	G1(1) G2(2) G3(1) G4(1)	4	G3 (subClassOf reasoning), G4 (GROUP_CONCAT aggregation)

and run behind each evaluation. Both queries run over the OWL-RL-materialised closure of the demonstrator; the full query set is part of the release package.

```
SELECT ?auth ?assigner ?assignee WHERE {
    ?run a it6:Run ;
         daimo:authorizedBy ?auth .
    ?auth a daimo:ExecutionAuthorization ;
          odrl:assigner ?assigner ;
          odrl:assignee ?assignee .
    FILTER (?run = <https://example.org/daimo-
        scenario/run-consumer-b>)
}
```

Listing 3: CQ-G3: authorisation and agreement for a specific run.

```
SELECT ?model ?metric ?value WHERE {
    ?eval a it6:Evaluation ;
          daimo:hasEvaluationContext ?ctx ;
          it6:hasMeasure ?m .
    ?m it6:metricType ?metric ;
       it6:metricValue ?value .
    ?run it6:hasEvaluation ?eval ;
         daimo:onDeployment/daimo:deploysModel ?model
              .
    FILTER (?ctx = <https://example.org/daimo-scenario
        /eval-context>)
} ORDER BY DESC(?value)
```

Listing 4: CQ-V3: ranking of models under a shared evaluation context.

Discussion

The central contribution of DAIMO is the articulation of five lifecycle relations that are usually documented separately: publication, policy, execution, provenance, and evaluation. This articulation is what makes governed AI-model exchange queryable across organisational boundaries. Relative to recent domain ontologies such as RePlanIT[23], GloSIS[24], and Woods et al.[30], DAIMO shares an engineering ethos of reuse, executable implementation, and CQ-based evaluation, but applies it to the catalogue-dataspace boundary. Relative to earlier machine-learning lifecycle ontologies, DAIMO shares the application domain but adds the governance layer needed for dataspace exchange. Three design features support this contribution. First, entailment verification over the OWL-RL closure complements consistency reasoning by detecting modelling-inappropriate alignments that a DL reasoner may accept without contradiction. Second, cross-class SHACL-SPARQL invariants are paired with a negative test bench, so every invariant is evaluated against a case built to violate it as well as against the conforming graph. Third, reuse is formalised rather than asserted informally: external commitments are represented as explicit alignments, while three tempting PROV-O alignments are deliberately excluded because they would introduce unintended typings.

Three lines of argument bound the limitations. The first concerns semantic depth. The I/O contract captures format and authentication method but not fine-grained semantic compatibility between model and consumer schemas, which is delegated to pipeline descriptions as Kipoi[46] and TFX[47] do for preprocessing and pipelines. The shared evaluation context reifies task-dataset-version-protocol-seed for single-metric comparisons; heterogeneous benchmarking protocols and composite metric sets remain open. The role model folds role type into role assignment, which OntoClean recommends splitting[48]. MLDCAT-AP makes a comparable compromise in its registered-user relation, which places fine-grained contextualisation in the future-refinement horizon, not at the level of a present design error.

The second line concerns the integration commitments and their effects on consumers who combine DAIMO with adjacent vocabularies. One provenance aggregation relation classifies recorded items as PROV activities; under RDFS reasoning, MLDCAT-AP evaluations grouped inside a cross-participant provenance record are therefore inferred as PROV activities even though MLDCAT-AP does not declare that subsumption. This does not affect the audit queries reported here, but a consumer combining DAIMO, MLDCAT-AP, and PROV-O may see surprising typings on evaluation records. Relatedly, DAIMO is not anchored to an upper ontology (BFO, GFO, DOLCE, UFO). The absence is intentional, because an integration profile is not a reference ontology. A coherent future step would be to accommodate the taxonomy to one of these upper ontologies without giving up principled reuse of the domain vocabularies that motivate DAIMO.

The third line concerns reproducibility conditions and external validation. Reported numerical results depend on a specific reasoning profile: inference=rdfs mode in pyshacl, necessary for implied subproperties to materialise before the conformance shapes are evaluated, and the OWL-RL closure materialised before the competency queries. This dependency is documented but is a condition a consumer must know to reproduce exact results. Expert-based validation is not part of the evidence reported here; unlike RePlanIT's interface evaluation and FATO's explicit use of domain experts, the present article reports executable validation rather than practitioner validation. A qualitative study with three practitioner profiles (an EDC or INESData engineer, an MLDCAT-AP/SEMIC maintainer, and an MLOps practitioner) is planned as the next phase of work. [TODO: Run the expert study: recruitment, semi-structured interview protocol, qualitative analysis, and publication of results.]

The adoption path is concrete even though a deployed pilot is not part of the present evidence. On the AI-asset side, DAIMO is designed to co-evolve with the SEMIC MLDCAT-AP dossier by preserving its model-description layer and adding only the dataspace-governance bridge. On the dataspace side, the natural integration host is Eclipse Dataspace Components: participant contexts can anchor aggregated provenance, and ODRL agreements already appear at the end of a DSP negotiation. On the governance side, DAIMO structures audit evidence around integrity records that a Gaia-X Trust Framework inspection, or an AI Act Annex IV traceability audit, would request: checksum, signer, timestamp, and linkage to a cross-participant bundle. These observations define three concrete validation paths: instantiation over a public MLDCAT-AP catalogue extract, co-deployment with an EDC connector on a federated pilot, and mapping of a high-risk AI Act use case against the six SHACL-SPARQL invariants. None of these has been executed yet; they define the short-term adoption roadmap rather than the present claims.

The relation between DAIMO and MLDCAT-AP deserves explicit delimitation. MLDCAT-AP supplies the AI-model description; DAIMO concentrates on the catalog-dataspace bridge. The four competency questions of the governance category cannot be answered with MLDCAT-AP alone, because they require an offering, a deployment, an execution authorisation, and a cross-participant provenance record. The two profiles are complementary.

Looking beyond the present artefact, DAIMO illustrates a pattern that is likely to recur wherever autonomous participants need to exchange high-value, policy-bound resources. The pattern has three recurring elements: an offering record reified above the resource itself, an authorisation reified above the permission, and a provenance bundle that crosses administrative contexts. Genomic datasets, clinical decision-support tools, digital twins, and industrial models all present analogous publication-invocation-auditing chains. DAIMO is AI-specific in what it reuses (MLDCAT-AP) but more general in the bridge constructs it adds, which can inform sibling profiles in adjacent domains.

Availability and FAIR compliance. DAIMO is prepared as a release package under CC-BY 4.0 (ontology and documentation) and Apache 2.0 (validation scripts). The package is designed to satisfy the FAIR principles[42]: findability through the persistent w3id.org/pionera/daimo namespace with per-version IRIs and Dublin Core metadata; accessibility through content negotiation across Turtle, RDF/XML, JSON-LD, and N-Triples; interoperability through reuse of W3C and community-standard vocabularies and the OWL 2 DL profile; and reusability through SKOS definitions and examples per term[41], SemVer versioning (with a deprecation policy based on owl:deprecated and skos:closeMatch), WIDOCO documentation[37], and SHACL shapes that act as a data contract. The artefact-availability appendix lists the deliverable items and their locations, and the full release procedure is documented in DEPLOYMENT.md so that future maintainers can reproduce the release process.

Conclusions

DAIMO contributes a governance bridge between the AI catalogue and the dataspace: offering, authorisation, deployment, derived artefact, and cross-participant provenance. The four governance-category competency questions make the gap concrete; they cannot be answered with any of the reused vocabularies alone. Entailment verification, a negative SHACL-SPARQL test bench, and the reproducible demonstrator provide complementary evidence for the profile. Future work follows from the bounded limitations: external expert validation, instantiation over a public MLDCAT-AP catalogue, refinement of I/O contracts toward schema compatibility, role-type/role-assignment separation following OntoClean[48], and extension of shared evaluation contexts to heterogeneous benchmarking protocols. The release package is prepared under CC-BY 4.0 (ontology) and Apache 2.0 (scripts).

References

1. European Union (2024) Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (AI Act). Official Journal of the European Union.
2. Eclipse Foundation. Eclipse Dataspace Components (EDC), control-plane documentation.
3. International Data Spaces Association (2024) Dataspace Protocol (DSP). W3C Community Draft.
4. SEMIC (2025) MLDCAT-AP 3.0.0: Machine Learning Data Catalogue Application Profile. European Commission.
5. Albertoni, A. et al. (2020) Data Catalog Vocabulary (DCAT) — Version 2. W3C Recommendation.
6. Iannella, R., Guth, S. and Steyskal, R. (eds.) (2018) ODRL Information Model 2.2. W3C Recommendation.
7. Lebo, P., Sahoo, S. and McGuinness, D. (eds.) (2013) PROV-O: The PROV Ontology. W3C Recommendation.

8. Mitchell, M. et al. (2019) Model Cards for Model Reporting. Proceedings of the Conference on Fairness, Accountability, and Transparency.
9. Arnold, M. et al. (2019) FactSheets: Increasing Trust in AI Services through Supplier's Declarations of Conformity. IBM Research Report.
10. Gebru, T. et al. (2021) Datasheets for datasets. Communications of the ACM, 64(12):86–92.
11. Soldatova, L.N. and King, R.D. (2006) An ontology of scientific experiments. Journal of the Royal Society Interface, 3(11).
12. Correa Publio, G. et al. (2018) ML-Schema: Exposing the semantics of machine learning with schemas and ontologies.
13. Keet, C.M., Fernández, J.D. and Panov, P. (2016) MEX vocabulary: A lightweight interchange format for machine learning experiments. ISWC.
14. Panov, P., Dzeroski, S. and Soldatova, L.N. (2008) OntoDM: An ontology of data mining. ICDMW.
15. Keet, C.M. et al. (2015) The data mining optimization ontology. Web Semantics, 32.
16. Souza, R. et al. (2019) Provenance data in the machine learning lifecycle in computational science and engineering. WORKS.
17. Schmidt, M. et al. (2021) FAIRness along the machine learning lifecycle using Dataverse in combination with MLflow. ICDE Workshops.
18. Pandit, H.J. and Esteves, B. (in press) Enhancing Data Use Ontology (DUO) for Health-Data Sharing by Extending it with ODRL and DPV. Semantic Web.
19. Franklin, M., Halevy, A. and Maier, D. (2005) From Databases to Dataspaces: A New Abstraction for Information Management. SIGMOD Record, 34(4):27–33.
20. Otto, B. et al. (2019) Reference architecture model for international data spaces. International Journal of Information Management, 45:182–199.
21. Cappiello, C., Gal, A., Jarke, M., Rehof, J. and Yang, Y. (2022) Data spaces: Design, deployment, and future directions. ACM Computing Surveys, 55(2):Article 46.
22. Werbrouck, J. et al. (2024) ConSolid: A federated ecosystem for heterogeneous multi-stakeholder projects. Semantic Web, 15:429–460.
23. Kurteva, A. et al. (in press) RePlanIT Ontology for Digital Product Passports of ICT: Laptops and Data Servers. Semantic Web.
24. Palma, R. et al. (in press) GloSIS: The Global Soil Information System Web Ontology. Semantic Web.
25. Baryannis, G., Jia, L. and Papadakis, E. (2025) FATO: The Food Allergen Traceability Ontology. Semantic Web.
26. Wu, J., Garcia, K., Mayer, S. and Albert, J. (2024) NutriLink: An Ontology for Linking Digital Receipts to Food Nutrition Information and Dietary Recommendations. Semantic Web.
27. Heuschkel, M., Höffner, K., Schmiedel, F., Labudde, D. and Uciteli, A. (2023) The ANthropological Notation Ontology (ANNO): A core ontology for annotating human bones and deriving phenotypes. Semantic Web.
28. Trypuz, R., Kulicki, P. and Sopek, M. (in press) Ontology of autonomous driving based on the SAE J3016 standard. Semantic Web.
29. Chari, S. et al. (in press) Explanation Ontology: A General-Purpose, Semantic Representation for Supporting User-Centered Explanations. Semantic Web.
30. Woods, C. et al. (in press) An ontology for maintenance activities and its application to data quality. Semantic Web.
31. Bareedu, Y.S. et al. (2024) Deriving semantic validation rules from industrial standards: An OPC UA study. Semantic Web, 15:517–554.
32. Ferranti, N. et al. (in press) Formalizing and Validating Wikidata's Property Constraints using SHACL and SPARQL. Semantic Web.
33. Robaldo, L. and Batsakis, S. (in press) On the interplay between validation and inference in SHACL. Semantic Web.
34. Penteado, B.E., Maldonado, J.C. and Isotani, S. (2023) Methodologies for publishing linked open government data on the Web. Semantic Web, 14:585–610.
35. Poveda-Villalón, M., Fernández-Izquierdo, A., Fernández-López, M. and García-Castro, R. (2022) LOT: An industrial oriented ontology engineering framework. Engineering Applications of Artificial Intelligence, 111:104755.
36. Grüninger, M. and Fox, M.S. (1995) Methodology for the design and evaluation of ontologies. IJCAI Workshop on Basic Ontological Issues in Knowledge Sharing.
37. Garijo, D. (2017) WIDOCO: A Wizard for Documenting Ontologies. ISWC Posters and Demonstrations.
38. Gruber, T.R. (1995) Toward principles for the design of ontologies used for knowledge sharing. International Journal of Human-Computer Studies, 43(5–6):907–928.
39. Brickley, D. and Miller, L. (2014) FOAF Vocabulary Specification 0.99. Namespace document.
40. SPDX Working Group (2022) SPDX Specification. Version 2.3.
41. Miles, A. and Bechhofer, S. (eds.) (2009) SKOS Simple Knowledge Organization System Reference. W3C Recommendation.
42. Wilkinson, M.D. et al. (2016) The FAIR Guiding Principles for scientific data management and stewardship. Scientific Data, 3:160018.
43. Brank, J., Grobelnik, M. and Mladenić, D. (2005) A survey of ontology evaluation techniques. Conference on Data Mining and Data Warehouses.
44. Knublauch, H. and Kontokostas, D. (eds.) (2017) Shapes Constraint Language (SHACL). W3C Recommendation.
45. Poveda-Villalón, M., Gómez-Pérez, A. and Suárez-Figueroa, M.C. (2014) OOPS! (OntOlogy Pitfall Scanner!). International Journal on Semantic Web and Information Systems, 10(2):7–34.
46. Avsec, Z. et al. (2019) The Kipoi repository accelerates community exchange and reuse of predictive models for genomics. Nature Biotechnology, 37.
47. Baylor, D. et al. (2017) TFX: A TensorFlow-based production-scale machine learning platform. KDD.
48. Guarino, N. and Welty, C.A. (2002) Evaluating ontological decisions with OntoClean. Communications of the ACM, 45(2):61–65.

Artefact availability

The reproducibility workflow does not require prior deployment of the persistent URL. Any user with Python 3.12 and the declared dependencies (rdflib, pyshacl, owlrl, owlready2) can run the four scripts locally and reproduce every report in reports/. The persistent URL and the DOI are additional FAIR requirements that complement but do not condition reproducibility.

Competency questions

R — Registration and publication (5)

1. CQ-R1: Which AI models has a given provider published as governed catalog assets?
2. CQ-R2: For a given model, which catalog offering registered it, by which publisher, and when? (requires subPropertyOf chain)
3. CQ-R3: Licence and usage policy of a model.
4. CQ-R4: I/O contract for the invocation interface of a published model.
5. CQ-R5: For each offering, what concrete ParticipantRole subclass does the providing agent hold? (requires subClassOf+)

D — Discovery and selection (4)

1. CQ-D1: Which models solve a given task in a given application domain?
2. CQ-D2: Which of those models are usable under a non-commercial policy pattern?
3. CQ-D3: Which models expose a service endpoint, and what authentication method does each require?
4. CQ-D4: Models reaching a minimum metric threshold under a shared evaluation context.

E — Execution and auditability (5)

1. CQ-E1: For a given catalog offering, what service endpoint, authentication, and I/O contract apply to invoking the underlying model? (requires entailment)
2. CQ-E2: For a given model deployment, which runs have been executed, by which agents, and under which execution authorisation?
3. CQ-E3: Implementation, algorithm, flow, and compute infrastructure used in a run.
4. CQ-E4: Audit evidence (hash, signer, timestamp) supporting a given run.
5. CQ-E5: Derived artefacts produced by a run, and under which authorisation.

V — Evaluation and reproducibility (5)

1. CQ-V1: Shared evaluation context of a given evaluation.
2. CQ-V2: Under a shared evaluation context, highest-scoring model for a metric.
3. CQ-V3: Ranking of models under the same evaluation context and metric.
4. CQ-V4: For a given model, which benchmark suites has it been evaluated on?
5. CQ-V5: Reproducibility artefacts backing an evaluation.

G — Governance bridge (4, DAIMO-specific)

1. CQ-G1: Offerings that include a given model.
2. CQ-G2: Deployments serving a model, with infrastructure and I/O contract.
3. CQ-G3: Authorisation (and agreement) that authorised a specific run.
4. CQ-G4: Full provenance bundle for a derived artefact across participant contexts.

SHACL shapes

The DAIMO SHACL layer contains nine structural nodal shapes, three conformance shapes over reused classes, and six cross-class SHACL-SPARQL invariants. The module shapes/daimo-shapes.ttl is itself declared as an independent owl:Ontology with its own owl:versionIRI under https://w3id.org/pionera/daimo/shapes/.

- Completeness shapes (9): AIAssetOfferingShape, ParticipantRoleShape, ModelDeploymentShape, IOContractShape, ExecutionAuthorizationShape, DerivedArtifactShape, SharedEvaluationContextShape, AuditEvidenceShape, CrossParticipantProvenanceRecordShape.
- Conformance shapes over reused classes (3): OfferInDAIMOShape requires odrl:assigner and odrl:target on every odrl:Offer; MachineLearningModelInDAIMOShape requires policy, title, and identifier on it6:MachineLearningModel; RunInDAIMOShape requires flow, algorithm, associated agent, and timestamp on it6:Run.
- Cross-class invariants (6): DerivationAuthorizationInvariant (INV-1), RunAgentAuthorizationInvariant (INV-2), DeploymentServiceInvariant (INV-3), AuthorizationTemporalInvariant (INV-4), OfferingPolicyTargetInvariant (INV-5), and OfferingAssignerInvariant (INV-6); all encoded as sh:SPARQLConstraint.

Table 9. Deliverable items of the DAIMO release package and their locations.

Item	Location
Persistent namespace	https://w3id.org/pionera/daimo; content negotiation configuration prepared under w3id-redirect/.htaccess. [TODO: Merge the PR on perma-id/w3id.org so that the namespace resolves via content negotiation.]
Versioned modules	Three independent ontologies (core, shapes, alignments) under https://w3id.org/pionera/daimo/ with explicit owl:versionIRI and owl:priorVersion
Public repository	[TODO: Create public GitHub repository pionera/daimo by running DEPLOYMENT.md and include the final URL here.]
Versioned archival (DOI)	[TODO: Archive the first release in Zenodo and include the resulting DOI here.]
Serialisations	Turtle, RDF/XML, JSON-LD, N-Triples (pre-generated by WIDOCO)
HTML documentation	WIDOCO 1.4.25 with WebVOWL; content negotiation via .htaccess under w3id-redirect/
Human-readable reference	ONTOLOGY-REFERENCE.md (class by class, with skos:definition, skos:example, identity criteria, OntoClean tags) and VALIDATION-MATRIX.md
Release metadata	CITATION.cff, .zenodo.json, CHANGELOG.md
Validation package	Structural checks, OWL-RL reasoning checks, OOPS! scan, and negative test bench
Maintenance governance	CONTRIBUTING.md (LOT Phase 4: issue triage, pull requests, SemVer, deprecation policy)

SPARQL queries

The twenty-three SPARQL queries for the competency questions are released with the validation package. The validator runs each query over the OWL-RL closure of the ontology-plus-example graph and verifies that each returns at least one row. Exact counts appear in Table 8. CQ-G2 returns two rows (one per endpoint of the REST/gRPC multi-protocol deployment), confirming that the multi-service deployment pattern is represented in the query results.

Glossary of abbreviations

URI conventions

DAIMO uses the canonical namespace https://w3id.org/pionera/daimo, with term identifiers of the form https://w3id.org/pionera/daimo#<Term> and versioned IRIs under https://w3id.org/pionera/daimo/<version>, linked to each other through owl:priorVersion. The satellite modules are independent ontologies with their own versioned IRIs: the SHACL layer under https://w3id.org/pionera/daimo/shapes/ and the alignment module under https://w3id.org/pionera/daimo/align (the latter declaring owl:imports on the base ontology). The example graph uses the local namespace https://example.org/daimo-scenario/, clearly separated from the canonical space to avoid confusion between demonstration identifiers and ontology terms. Content negotiation resolves the same identifier to HTML, Turtle, RDF/XML, JSON-LD, or N-Triples depending on the Accept header; the corresponding .htaccess configuration is prepared under w3id-redirect/.htaccess pending integration in perma-id/w3id.org.

Table 10. Abbreviations used in this article.

Abbrev.	Expansion
AI Act	Regulation (EU) 2024/1689 on artificial intelligence
BFO	Basic Formal Ontology
CQ	Competency Question
DAIMO	Dataspace AI Model Ontology
DCAT	Data Catalog Vocabulary
DCAT-AP	DCAT Application Profile
DL	Description Logic
DOI	Digital Object Identifier
DPV	Data Privacy Vocabulary
DSP	Dataspace Protocol
EDC	Eclipse Dataspace Components
FAIR	Findable, Accessible, Interoperable, Reusable
FOAF	Friend Of A Friend vocabulary
GFO	General Formal Ontology
I/O	Input / Output
LOT	Linked Open Terms methodology
MLDCAT-AP	Machine Learning Data Catalogue Application Profile
OOPS!	OntOlogy Pitfall Scanner
ODRL	Open Digital Rights Language
OWL	Web Ontology Language
OWL-RL	OWL 2 RL profile / rule-based reasoning
PROV-O	PROV Ontology
RDF	Resource Description Framework
RDFS	RDF Schema
REST	Representational State Transfer
SHACL	Shapes Constraint Language
SKOS	Simple Knowledge Organization System
SPARQL	SPARQL Protocol and RDF Query Language
SPDX	Software Package Data Exchange
TTL	Turtle (RDF serialisation)
UFO	Unified Foundational Ontology
WIDOCO	Wizard for Documenting Ontologies